

AI-Based Code Style Enforcement with Code Llama

BSc (Hons) Computer Science Dissertation

Mediterranean College
University of Derby
May 2026

Student's name-surname:	Dimitrios Zerkidis
Student's ID No:	100778080
Module:	6CM995 Dissertation
Supervisor:	Ioannis Tsioulis

Abstract

Move smart contracts on the Sui blockchain lack automated tools for enforcing code style. Naming, documentation, import ordering, indentation, spacing, and function signature formatting are left to developer judgment. This study tests whether Code Llama, an open-source coding model from Meta, can detect style violations in Move and give useful corrective feedback without fine-tuning. A dataset of 368 Move code samples was collected from public Sui ecosystem repositories, each annotated with a style error category and a reference feedback comment across six categories, and reviewed by a Sui Foundation representative. Each sample was run through Code Llama locally via Ollama using a fixed prompt and scored against the reference feedback. Strict accuracy, requiring a match with the specific Move convention, was 29.9 per cent. Lenient accuracy, which credits correct identification of the issue area, was 86.1 per cent. Results varied with language-specificity: categories where the correct answer is the same across languages scored well, while categories that depend on Move-specific conventions scored poorly. The dataset is the first annotated corpus of Move style errors from production code and establishes a baseline for downstream fine-tuning work.

Keywords: Move, Smart Contracts, Code Style, Code Llama, Sui, LLM

Acknowledgements

Thanks to my supervisor, Ioannis Tsioulis, for the guidance throughout this project and for the patience with the many revisions along the way. Thanks also to the Sui Foundation representative who reviewed the dataset and confirmed the annotations against ecosystem practice.

Most of all, thanks to my parents for their support and encouragement throughout my studies.

Table of Contents

Abstract	2
Acknowledgements	3
Table of Contents	4
1. Introduction	5
1.1 Aims and Objectives	6
1.2 Research Questions	7
1.3 Project Boundaries	7
2. Literature Review	9
2.1 Smart Contracts and the Move Language	9
2.2 Code Style and Maintainability	11
2.3 Large Language Models for Code	13
2.4 Current Tools and the Research Gap	15
2.4.1 Move-specific analysis tools	15
2.4.2 The style enforcement gap	16
3. Methodology and Implementation	20
3.1 Research Design	20
3.2 Style Rules Definition	21
3.3 Dataset Construction	22
3.3.1 Repository selection and sampling	22
3.3.2 Annotation and validation	23
3.4 Code Llama Setup and Prompt Design	26
3.5 Evaluation Approach	27
4. Results and Evaluation	30
4.1 Dataset Overview	30
4.2 Quantitative Results	30
4.3 Qualitative Findings	32
4.4 Case Studies	34
Case Study 1: Correct Detection with Generic Advice	34
Case Study 2: A Complete Failure	35
4.5 Discussion	36
4.5.1 Answering the research questions	36
4.5.2 Comparison with prior work	37
5. Conclusion	39
5.1 Summary of Contributions	39
5.2 Limitations	39

5.3 Future Work.....	41
Self-Reflection	43
References.....	45
Appendix A : Ethical Form.....	49
Appendix B : Project Plan Gantt Diagram.....	50

1. Introduction

In the Sui blockchain's smart contracts, using the Move programming language eliminates certain programming errors instead of just warning against them, and enforces strict correctness requirements through its type system. Like all programming languages, Move depends on the developer to choose their own style choices for the code: how to name functions, how to order the imports of a file, how to document a public interface, and how to lay out function signatures. Although the style of the code does not affect the ability of code to compile or run correctly, the style chosen by the developer will ultimately impact the readability, reviewability, and maintainability of the code.

Many established programming ecosystems have automated tools to assist with enforcing style choices. For example, Python has PEP 8 and dozens of linters, Rust has Clippy, and Solidity has Solhint. However, there are no such tools available for the Move ecosystem. Although the Sui ecosystem has a code formatter that will automatically insert spaces, the Sui ecosystem does not currently have a code analysis tool to check for naming convention adherence, completeness of documentation, or order of imports. The Sui ecosystem is growing faster than the development of tools for maintaining code readability, and the developers of Move code today are likely to be the developers of Move code tomorrow, which means that the style choices made today by developers will impact the Sui codebase for an extended period.

The research question is whether a general-purpose large language model without any adaptations for Move can be of benefit in filling this gap in the Move code ecosystem. Code Llama is a coding model from Meta (Rozière et al., 2023) that can be used locally on developer machines via Ollama. While this tool was developed on multiple programming languages, there is currently no empirical evidence showing that it has the ability to work on Move language code, because there is no existing dataset of Move style errors available that could serve as an evaluation framework. This is the first of two focuses of this thesis.

It creates such a dataset, consisting of 368 samples of real-world Sui network Move code, each assigned a specific error class with associated comments from the Sui Foundation representative.

The second focus is to evaluate the performance of the Code Llama system against Move code, using the dataset created as a baseline. The collected dataset provided an initial opportunity to create and evaluate the Code Llama analysis tool and to contribute to that body of knowledge as well.

This study also represents a larger Sui Academic Research collaborative partnership, with the overall goal of developing a Move code analysis tool using an improved version of Code Llama. The different partners will also be gathering information for other common forms of code error, including security issues, syntactical errors, and semantic errors. The scope of this thesis is to focus on the code styling error category and establish the baseline for the Move code error collection and comparison to the improved version of Code Llama in the future.

The rest of this document has been structured with these two contributions in mind. Chapter Two presents an overview of relevant literature regarding smart contracts, Move, code styles and maintainability, LLMs for code, and the current state of tooling for Move analysis. Chapter Three describes how the dataset was built and how the Code Llama evaluation was performed. Chapter Four contains the results in aggregate and by code style, along with a small group of case studies demonstrating the patterns that emerged during the analysis. Finally, Chapter Five summarises this study, outlines its limitations, and discusses opportunities for future research based on the findings.

1.1 Aims and Objectives

The aim of this study is to evaluate whether Code Llama can effectively detect style errors in Move smart contracts and generate feedback that is accurate, specific, and useful to developers.

This aim is pursued through five objectives:

1. Define a set of style rules for Move code in the Sui ecosystem, grounded in official documentation and community conventions.

2. Construct and annotate a dataset of 368 Move code samples from real Sui ecosystem repositories, each labelled with a style error category and a reference feedback comment.
3. Configure and run Code Llama on the full dataset using a consistent prompt template, capturing its output for each sample.
4. Evaluate the model's outputs against the Sui-reviewed feedback, assessing category identification accuracy, feedback accuracy, and overall correctness for each of the six style categories.
5. Discuss the implications of the findings for the practical use of large language models in Move code quality workflows, and identify where the approach succeeds and where it falls short.

These objectives move from definition through construction to evaluation and interpretation. The dataset produced by objectives one and two is a contribution in its own right, as no equivalent resource currently exists for Move style analysis. The evaluation conducted through objectives three and four provides the empirical evidence from which the conclusions in objective five are drawn.

1.2 Research Questions

The analysis in this research is structured around four questions.

The first question examines whether Code Llama can recognise Move style errors. The second question examines how Code Llama's feedback compares to the feedback reviewed by Sui, because recognising an issue is useful only if it is also assessed accurately.

The third question asks which categories of style errors Code Llama is best at recognising and assessing.

The fourth question considers how AI-based style enforcement would be used in practice. This question allows the empirical data to be connected with the larger Sui ecosystem and explores whether Code Llama is a potential interim solution for a developer community not currently supported by automated style-checking tools.

1.3 Project Boundaries

This study analyses code styles in Move smart contracts deployed to the Sui blockchain. The study does not include syntax errors, which the Move compiler detects at build time, or semantic errors, which are verified by the Move Prover. The evaluation assesses six styles of Move code as identified in the Methodology chapter and is restricted to these styles.

The Code Llama model is used without fine-tuning and is not compared against other models. The research does not evaluate fine-tuned Code Llama models or assess performance through prompt optimisations. Since the key question of this study is to determine what a general-purpose model will produce in its unmodified state, the findings cannot be extrapolated to fine-tuned models or to the language models of other providers.

The dataset consists only of Move code from Sui ecosystem repositories and does not include code from other platforms that use the Move language. Different dialects of Move exist on different platforms, and mixing samples from more than one dialect would create unwanted variation that would be counterproductive to the analysis of the categories defined in this study. The table below summarises the primary risks identified in this study and how they will be managed.

Table 1.1: Project risks and mitigations.

Risk	Likelihood	Impact	Mitigation
Annotation bias from single annotator	Medium	Medium	Clear category definitions with documented examples; consistent annotation format applied across all samples
Model training data may include some Sui repositories	Low	Low	Noted as limitation; relative performance differences between categories remain valid regardless
Prompt produces inconsistent output lengths	Low	Low	Fixed prompt template; automated pipeline ensures identical conditions for all samples
Dataset not fully representative of Move style patterns	Medium	Medium	Sourced from multiple mature repositories across DeFi and infrastructure domains

2. Literature Review

2.1 Smart Contracts and the Move Language

The first mention of smart contracts (digital contracts that enforce their own terms) was made by Szabo in 1997, prior to the actual development of a platform upon which such contracts could be run. This platform was later developed with the blockchain, which was developed by Nakamoto in 2008, and can be thought of as a distributed ledger technology. Smart contracts are computer programs executed on a blockchain and triggered when specific criteria are met, as noted by Buterin (2014). Unlike traditional computer programs, smart contracts have limited or no ability to be modified once deployed, and the only way to alter an existing smart contract is through a series of complex actions and by creating and migrating to a newly created smart contract. The irreversibility of smart contracts creates the necessity to ensure that the original code is of the highest quality, as seen with the incident involving the Parity Wallet, where the single flaw in a library contract resulted in approximately \$280 million in frozen assets (Parity Technologies, 2017). Furthermore, Chainalysis (2022) reported that approximately \$3.8 billion was lost to hacking and other forms of theft on various cryptocurrency platforms in 2022. The same risk and vulnerability apply to other blockchain platforms where contracts run without an off switch. Solidity, the most widely-used programming language for creating smart contracts on Ethereum, was created to allow JavaScript developers the ability to create smart contracts quickly by writing code similar to their existing experience. However, building this language created trade-offs as Solidity allows for the creation of potentially dangerous patterns known as External calls, which allow a third-party contract to call the calling contract before the state of the calling contract is updated. It is the responsibility of the developer to prevent these types of vulnerabilities from occurring as there is no enforcement by the language. Giatzis et al. (2025) present a comparison of the architecture between the Solidity language and a new language called Sui Move and provide an argument as to why the two languages have a fundamentally different architecture due to the difference in syntax, data models, mechanisms of dispatch and methods of controlling security. Their comparison clearly illustrates that some of the design choices made in Move lead to coding patterns that cannot be migrated from one environment to another. As a result, both conventions and tooling for coding in Move will be considerably different from that of Solidity.

The programming language known as Move was developed by Meta in conjunction with the Diem blockchain project, to address the problems associated with Solidity. The primary design element behind Move is the definition of "resource types" (Blackshear et al., 2019). A resource in Move can be said to be a type of object that is distinct from a traditional object in that it cannot be copied or lost through an action that would usually cause the resource to become invalid. Move enforces a strong type system which prevents the double-spending of tokens and losing them via bugs due to bad programming practice. Bartoletti et al. (2025) compare six smart contract languages in terms of security, usability, and readability, and find that Move's linear type system provides a strict application of must-move semantics. This is what places Move among the safest smart contract languages. Patrignani and Blackshear (2021) have formalised these standards and shown that the type system of Move can be used to establish (and contains) what they call "robust security," which refers to the ability for modules with well-typed elements to not be compromised by the use of code from any of the other modules (including those modules developed by adversaries). As a result of this modular design, developers will need to think carefully about how to name and structure their code as clearly as possible in order for both the developers and anyone reading the code to be able to clearly understand the boundaries of each of the modules.

Sui uses Move for its smart contracts, and it has extended Move to allow for the creation of an object-centric data model. In this setup, every object in the chain has a unique identifier and an identified owner (Mysten Labs, 2022). The Sui consensus mechanism uses the Narwhal mempool and the Tusk BFT protocol (Danezis et al., 2022), which enables the ability for transactions that involve owned objects to be performed in parallel. This allows the Move ownership model to act both as a correctness and performance mechanism. The Sui mainnet was launched in May 2023, and since that time, there has been significant growth in the number of developers building decentralised finance (DeFi), gaming, and collectibles applications. As this market continues to evolve, it will be critical that code quality is maintained across the rapidly increasing number of developers and the growing number of projects. Move's type system provides a solid foundation for developers building code with a good degree of correctness, but it does not dictate how developers should structure their imports, name their methods, or document their code. Each of these items has a direct impact on how readable and maintainable the resulting code will be, and the way they are structured is currently left solely to the developer's discretion.

Table 2.1: Architectural comparison of Solidity and Sui Move (adapted from Giatzis, Papangelou and Georgiadis, 2025).

Feature	Solidity (Ethereum)	Sui Move
Data model	Account-based with key-value storage	Object-centric with unique IDs
Asset representation	Numeric balances in contract storage	First-class resources with ownership metadata
Function dispatch	Dynamic (resolved at runtime)	Static (resolved at compile time)
Reentrancy	Possible; mitigated by developer patterns	Structurally impossible; static dispatch prevents it
Overflow / Underflow	Built-in since Solidity v0.8; absent in earlier versions	Built-in arithmetic safety; transaction aborted on detection
Asset safety	Developer responsibility; assets can be copied or lost	Enforced by type system; resources cannot be copied or implicitly discarded
Parallel execution	Sequential; single global state updated per transaction	Owned objects processed in parallel; shared objects sequential
Access control	Developer-implemented via modifiers and patterns	Integrated into language via object ownership and abilities
Formal verification	External tools (limited scope, not fully automated)	Move Prover (bytecode-level, integrated into toolchain)
Ecosystem maturity	Established since 2014; extensive tooling and libraries	Mainnet since May 2023; growing but limited tooling

2.2 Code Style and Maintainability

Software coding conventions are defined by the rules for formalising code including naming, formats, indentation, and documentation. Coding conventions do not affect how a program runs but do affect how efficiently a human can interact with that program.

Developers spend their day reading code because software development is predominantly a reading task. Minelli, Mocci and Lanza (2015) show that programmers primarily read, navigate, and comprehend current software during their workday, which means anything that makes code harder to read costs everyone after the author has written the code.

For example, if a codebase follows snake_case for all function identifiers and PascalCase for all type identifiers, then it is clear to a developer reading that code that they do not need to stop and verify what type of identifier they are currently inspecting. However, if the same codebase contains identifiers that mix or vary in naming conventions, the developer must

exert additional effort to identify what type of identifier they are currently reviewing. Lawrie et al. (2006) studied identifier naming and demonstrated that providing a consistent and descriptive name for an identifier substantially shortens the amount of time required to comprehend code.

Formatting and indentation provide insight into the overall structure of code and give a clear indication of how many levels down from the root the code is nested. In addition, consistent formatting and indentation reduce the visual clutter associated with operators and expressions. Although any single example may not provide a substantial benefit, the cumulative effect across an entire codebase can be substantial.

The purpose of documentation is to provide information about additional topics that may not be addressed in the code itself. Comments and documentation strings express the purpose of a piece of software and why it was written that way. For smart contracts, leaving details about how they work, as well as examples of how they are used, gives auditors the ability to easily verify whether the software performs exactly as intended. This is important because auditors cannot rely solely on what the code does. They must have a reference point.

The findings of Buse and Weimer (2010) and Posnett, Hindle and Devanbu (2011) provide evidence of a strong correlation between how readable a piece of code is, as shown by the automated readability metrics created by Buse and Weimer, and its defect rate. Buse and Weimer found that code that is easier to read has a lower number of defects compared to code that is more difficult to read. Posnett, Hindle and Devanbu (2011) found that style guide indicators, such as uniform indentation and identifier length, can also predict how quickly developers are able to interpret a piece of code. Thus, regardless of whether lower readability results in more defects, the conclusion remains that there is a positive return on investment for improving readability and using style guides to establish common standards.

A commonly known example of a style guide in software development is PEP 8 (van Rossum, Warsaw and Coghlan, 2001), which is widely used by the Python community. The PEP 8 style guide provides developers with guidelines for naming, formatting and structuring their Python code. In addition to style guides, there are many tools available that automatically format code, such as the Black formatter for Python or the Prettier formatter for JavaScript, in accordance with predetermined formats, thereby taking the formatting decisions away from developers. The use of automated linters such as ESLint

and Pylint allows a developer to easily identify coding style violations. Many of the rules that these linters identify can also be corrected automatically. Integrating linters into a continuous integration workflow makes it possible for projects to consistently produce code that follows the same style, whereas previously developers had to actively work to maintain uniformity across their projects.

Boogerd and Moonen (2008) conducted an empirical study on coding standards and concluded that a small number of coding standard rules is responsible for the majority of coding standard violations. The authors proposed that targeting enforcement of the most common coding standard violations would return the greatest value to an organisation and its developers. Kupari et al. (2025) support this conclusion with data from more than 1,000 publicly available Java projects hosted on GitHub. The authors found that the coding standard violations within these projects were very stable over time, with a predominant focus on Javadoc and naming violations, suggesting that style uniformity does not correct itself without the assistance of an automated linter.

The Move programming language includes built-in basic formatting features, such as the normalising of whitespace and basic layout structure. However, Move does not have a full-featured linter to verify that the developer is uniformly using documentation comments and naming conventions in line with the defined coding standards for the language. This is the area in which this research study focuses.

2.3 Large Language Models for Code

In the last several years, large language models (LLMs) trained on source code have become increasingly popular for use in software development. These LLMs recognise common patterns from large sets of text and source code data, and then use those patterns to respond to prompts with generated, modified, or transformed code.

The general-purpose LLM GPT-3, trained on mostly natural language, generated code in ways that surprised everyone (Brown et al., 2020). The development of Codex, which was trained on public source code repositories, led to the creation of GitHub Copilot, the first significant AI coding assistant available to the public (Chen et al., 2021). GitHub Copilot proved that LLMs could help programmers with daily tasks such as creating functions, auto-generating boilerplate code and converting natural language into executable code.

Studies have also revealed limitations. Pearce et al. (2022) found that a significant portion of Copilot's suggested code contained security vulnerabilities. The key conclusion is that LLMs are a valuable tool but must undergo human review prior to production use. They can be effective as assists but not as substitutes for programmer judgment.

In August 2023, Meta released Code Llama, an open-source model based on the Llama 2 architecture (Touvron et al., 2023; Rozière et al., 2023), trained on around 500 billion tokens of code from many popular programming languages. The models are available with 7 billion to 70 billion parameters in three types: the base model, the Python-specialised model, and the instruction-following model for interactive use. Because it is open-source, Code Llama can be downloaded and run on local hardware without commercial API services.

Performance across programming languages differs. Languages with large amounts of training data, such as Python, Java, C++, and JavaScript, see positive results, whereas languages with less training data do not perform as well since there are not enough examples for the model to recognise language-specific patterns. Since Move is a newer and somewhat specialised language, there is probably not much Move code in the training dataset, so the model has likely had few opportunities to learn Move-specific patterns.

Research has found that different bugs can be corrected with different levels of success. Xia, Wei and Zhang (2023) found that models achieve high success on bugs with clearly identifiable mechanical patterns, such as incorrect operators, missing checks, and structural errors. When correcting bugs that require understanding of program semantics or developer intent, models had more difficulty.

There is a related aspect to enforcing style in code. Many instances of style violations can also be repaired using simple pattern-matching techniques with little or no understanding of what the program is doing, such as replacing camelCase with snake_case, adding spaces around operators, or reordering imports. Since style violations are similar across all programming languages, it is likely that a model trained primarily on languages other than Move will still produce reasonably accurate results when correcting styles in Move files.

However, certain style violations require more than pattern matching. For example, to create documentation for a particular function, the model must be able to understand the purpose of that function as well as how to use the parameters and the return values. This is a comprehension-based task, and the model's lack of familiarity with the language will

likely lead it to generate documentation that lacks an appropriate level of detail and understanding. One of the primary goals of this study is to show how Code Llama manages the differences between comprehension-based and pattern-based tasks in Move.

Prompt engineering is also critical to output quality. How a prompt is structured greatly affects the resulting output, and the same task under a different prompt structure, level of detail, or format will produce different outputs. Prompt design is discussed in the Methodology chapter.

The empirical study most closely related to this work is that of Karanjai et al. (2025), who created a unified agentic framework for repairing vulnerabilities in both Solidity and Move smart contracts, and who also tested multiple LLMs for repair tasks across the two languages. Their study demonstrated that the lack of available training material related to Move creates significant performance disparities. For example, models that performed well on Solidity repair tasks did not perform as well on equivalent Move repair tasks. The Karanjai et al. (2025) study limited its focus to security vulnerabilities, while this study examines code style enforcement issues and measures the quality of natural-language feedback.

The results are similar across the general literature on LLMs for code. The work of Sobania et al. (2023) and the work of Prenner, Babii and Robbes (2022) both involved the evaluation of models against common bug repair benchmarks, and the results demonstrated strong success at making mechanical repairs but a significantly lower success rate at making repairs that require semantic understanding rather than surface pattern matching. Zhang et al. (2024) systematically reviewed the literature on LLMs as tools for the automation of program repair and concluded that the two most important variables influencing LLM performance were the composition of training data and the method for structuring prompts, particularly for programming languages that are poorly represented in the training corpus, as is the case with Move.

2.4 Current Tools and the Research Gap

2.4.1 Move-specific analysis tools

Move's toolchain has many foundational elements. The Move compiler performs deep static analysis of code through the type system to find coding errors at compile time rather than at run time. The unit testing framework for Move provides unit testing for Move

contracts, while the Sui CLI allows Move contracts to be built, tested, and deployed as one entity.

In comparison, the tools available for style enforcement are quite limited. The Move formatter normalises whitespace and indentation, that is, the way code is displayed, but it does not verify that variable names follow the format set out in the official Move documentation, that documentation exists for all functions, or that imports are correctly sorted in accordance with the official documentation.

In the last few years there has been a significant increase in the academic literature covering Move-specific tools for analysis, but the work conducted so far has focused on developing tools that aim to improve security when developing contracts. For example, Song et al. (2024) introduced a Move static analysis framework called MoveScan, which detects four classes of security vulnerability based on data flow analysis of Move contracts and patterns defined in the Move documentation. Zhang et al. (2025) analysed four classes of critical vulnerability that are only of concern in the context of Move, due to the way Move manages resources and ownership. Luo et al. (2025) developed the MoveScanner static analysis framework for automating security assessments on Move contracts. The Beosin Blockchain Security team released MoveLint (2023), an open-source linting tool that targets the identification of security properties in Move contracts. Sun et al. (2023) proposed a method for analysing Move contracts for resource-flow vulnerabilities by tracking when and how resource objects are used by functions and how they are passed between modules. Keilty (2023) explored model checking and gas optimisation in Move contracts, which is one of the few academic studies to examine contract quality rather than focusing solely on the security aspect of Move contracts. The Move Prover (Zhong et al., 2020) is an example of how formal verification can be used to guarantee correctness in Move contracts by using machines to check formal specifications. However, none of these tools provide style guarantees. All tools currently developed for Move contract analysis have been developed along the security axis, and style enforcement in Move analysis needs additional research and development.

2.4.2 The style enforcement gap

Giatzis and Georgiadis (2025) point out a similar gap to that identified by Keilty, but from a different perspective. They conducted a systematic ontological review of the ecosystem of Sui Move contracts and identified an issue known as code opacity. Specifically, many

contracts deployed on the Sui platform do not have publicly available source code. This means that there is no way to verify the design or quality of the deployed contracts through visual inspection of the source code. The authors also conclude that the analysis techniques currently available have significant limitations in addressing the unique object-oriented semantics of Move, and they argue that current analysis methods cannot effectively analyse the style of Move contracts. These findings highlight the increasing importance of ensuring the readability and style consistency of Move source code once it becomes available, because the only way collaborators and auditors can verify the intent of Move contract code is if it is readable.

The differences between Move and more established ecosystems are significant. Solidity has static analysis tools such as Slither (Feist, Grieco and Groce, 2019) for security, and style-checking tools such as Solhint and Prettier for formatting. Rust has similar tools, such as Clippy, which checks for performance, style, and common mistakes. The Python ecosystem has numerous tools and guidelines related to code quality. These tools and guidelines did not appear overnight. They are the result of many years of work and commitment from many people within the community. Sadowski et al. (2018) documented their experience at Google developing static analysis tools and state that the success of static analysis tools depends on the ability of developers to achieve low false positive rates and to integrate the tools into their existing work processes. The same concept would apply to any style tools developed for Move. Through a systematic literature review of blockchain and smart contract development techniques, Vacca et al. (2021) identify tool maturity as a barrier to adoption for newer ecosystems, and this finding directly represents where Move currently stands in terms of tool maturity.

The following table illustrates the gap by comparing available Move tools against equivalent tools from Solidity and other mature ecosystems.

Table 2.2: Comparison of available Move analysis tools against equivalent tools from other ecosystems.

Tool	Language	Primary Purpose	Style Checking
MoveScan (Song et al., 2024)	Move	Security vulnerability detection	No
MoveScanner (Luo et al., 2025)	Move	Security risk analysis	No

MoveLint (BeosinBlockchainSecurity, 2023)	Move	Security linting	No
Move Prover (Zhong et al., 2020)	Move	Formal verification	No
Sun et al. (2023) resource-flow analyser	Move	Vulnerability detection	No
Keilty (2023) model checker	Move	Model checking and gas optimisation	No
Slither (Feist et al., 2019)	Solidity	Security analysis	No
Solhint	Solidity	Style and security linting	Yes
Clippy	Rust	Style, performance, correctness	Yes
ESLint	JavaScript	Style and correctness	Yes

Developers who use Move lack the means to apply code style guidelines automatically: they can conduct code reviews but they do not scale, they can create custom scripts that will require substantial work to develop and maintain, and allowing for inconsistency will reduce the quality of their code over time. The absence of style tooling is not unique to Move in a historical sense; Iuliano et al. (2024), De Baets et al. (2024), and Hejazi and Lashkari (2025) each provide surveys of smart contract vulnerability detection tools and note that tooling maturity for any given language tracks closely with ecosystem age, with security tools typically arriving before style tools in every ecosystem that has developed them.

Language models might provide opportunities to improve this deficiency. If a model like Code Llama can detect style violations in Move code, it provides useful results immediately while the ecosystem develops dedicated tools for this purpose. In addition, language models may also be helpful in providing natural language explanations for code violations where a specific linter might not yet function.

While Karanjai et al. (2025) have evaluated multiple LLMs on Move vulnerability repair, no study has examined LLM performance on Move code style enforcement specifically. The only evaluations available today utilise widely-used programming languages for which there are already established performance benchmarks such as HumanEval (Chen et al., 2021). Move has no equivalent benchmark, and its unique characteristics, including resource types, ability constraints, and Sui's object ownership model, do not allow one to assume that performance on other languages would transfer.

With this study I intend to create an empirical resource to evaluate Code Llama's ability to check code style, which will fill a void in the current literature and create new evidence of how a general-purpose coding model can assist in ensuring code style across an underrepresented programming language.

3. Methodology and Implementation

3.1 Research Design

The fundamental question this study asks is whether Code Llama, applied without modification or fine-tuning, can detect and provide useful feedback on style violations in Move smart contracts. That framing matters for the design. This is an evaluation study, not a training study. There is no model development, no fine-tuning pipeline, and no attempt to improve Code Llama's performance through adaptation. The study takes the model as it exists and measures what it can do on a purpose-built dataset representing real Sui ecosystem code.

This design choice reflects a practical reality, most developers who might adopt an LLM-based style tool will be running a general-purpose model and not a fine-tuned one. Fine-tuning requires labelled training data, compute resources, and maintenance as the model is updated. A general-purpose model running locally through a tool like Ollama can be adopted with minimal friction. Understanding whether that zero-adaptation baseline is sufficient is therefore a more practically relevant question than what a fine-tuned model might achieve, and it provides a lower bound on performance that any fine-tuned variant should exceed.

Xia, Wei and Zhang (2023), in their study of LLMs for automated program repair, established a methodological framework that has influenced subsequent evaluation work: a fixed annotated dataset, a consistent prompting strategy, and comparison of model output against Sui-reviewed feedback. Their finding that general-purpose models handle pattern-based corrections well but struggle with semantics-dependent tasks informed the hypothesis underlying this study, namely that pattern-matching style errors such as spacing and naming may yield better results than comprehension-dependent tasks such as documentation generation. The present study follows the same evaluation structure adapted for a style enforcement context rather than a bug repair one.

The study proceeds in three phases. In the first, I defined the style categories and built a dataset of Move code samples annotated with category labels and reference feedback. In the second, I ran each sample through Code Llama via a standardised prompt and captured the output. In the third, I compared the model's output to the Sui-reviewed feedback for

each sample, scoring each response and identifying patterns across categories. The following sections describe each phase in detail.

3.2 Style Rules Definition

Before any code samples could be collected, I needed a clear definition of what constitutes a style error in Move. This is not a trivial question. Style conventions are partly codified in official documentation and partly emergent from community practice visible in mature repositories. For this study, I defined six categories of style error, derived from the Sui Move documentation, community conventions, and the patterns observable across well-maintained Sui ecosystem projects. Grounding the categories in published software engineering principles, as Martin (2008) and McConnell (2004) do for naming and formatting respectively, provides a basis for the choices that goes beyond local convention.

The first category is **InconsistentNaming**, which covers violations of Move's identifier naming rules. In Move, functions and variables use `snake_case`, struct types use `PascalCase`, and error constants follow the pattern `E_UPPER_SNAKE_CASE`. Martin (2008) argues that consistent naming is one of the most impactful readability improvements a developer can make, and Move's naming conventions are specific enough that violations are unambiguous. A constant defined as `ENotOneTimeWitness`, for instance, should be written as `E_NOT_ONE_TIME_WITNESS`; the camel-prefix form that some developers carry over from other languages violates the ecosystem standard clearly and consistently.

The second category is **Spacing**, which covers whitespace violations around operators, commas, and brackets. Clean spacing is a surface-level concern, but McConnell (2004) describes consistent spacing as one of the easiest style improvements to apply and one of the most immediately noticeable when violated. An expression written as `copy+ drop`, for example, should read `copy + drop`; the missing spaces force the reader to parse operator precedence manually in a way that properly spaced code does not.

The third category is **IncorrectIndentation**. Move uses four-space indentation throughout, and nested blocks should be indented consistently relative to their containing scope. Indentation communicates structure: code with inconsistent indentation is harder to navigate because the visual hierarchy no longer maps reliably to the logical hierarchy of the program. This is particularly relevant in Move, where nested match arms, assert statements, and function body blocks can span many lines.

The fourth category is **MissingDocumentation**, which refers to the absence of doc comments on public functions. In Move, the convention is to use triple-slash doc comments (`///`), placed immediately above function declarations. These comments serve two purposes: they explain function behaviour for developers reading the code, and they provide the information that security auditors need to verify that a contract does what it claims to do.

The fifth category is **InconsistentImportOrdering**. Move convention groups imports by namespace, placing standard library imports (those from the `0x1` address) before Sui framework imports (`sui :`), which in turn precede third-party and local package imports. Within each group, imports should appear in alphabetical order. Disorganised imports are harder to scan when checking whether a dependency is present, and they make merge conflicts more likely in codebases with multiple contributors.

The sixth category is **InconsistentFunctionSignature**, which covers formatting issues in function declarations. Long function signatures should use consistent line breaks and alignment to separate the function name, type parameters, parameters, and return type onto distinct lines. A signature formatted without line breaks becomes difficult to parse when the parameter list extends beyond a comfortable reading length.

These six categories were chosen because they represent the most frequently occurring style issues found when browsing Sui ecosystem repositories, they have unambiguous correct forms against which model output can be evaluated, and they span the range from pure pattern-matching tasks to tasks that require some contextual understanding of the code's purpose.

3.3 Dataset Construction

3.3.1 Repository selection and sampling

The dataset was built by sampling Move code from open-source repositories in the Sui ecosystem. The starting point was the Electric Capital crypto-ecosystems taxonomy (Electric Capital, 2024), an open-source mapping of blockchain ecosystems to their GitHub repositories, maintained as a community resource and used widely in blockchain research as a systematic way to identify the full scope of projects building on a given platform. For the Sui ecosystem, the taxonomy lists hundreds of repositories spanning DeFi protocols, NFT platforms, gaming applications, and infrastructure tools. Using it as the starting point

provided a systematic and reproducible basis for repository selection rather than an ad-hoc search.

From the full list of Sui ecosystem repositories, I focused on those with a significant volume of Move code and active maintenance histories. Infrastructure projects, core protocol libraries, and established DeFi applications were prioritised because these repositories tend to have large, well-structured codebases where style issues are both present and meaningful in context. Toy examples and proof-of-concept repositories were excluded on the grounds that style violations in experimental code are less representative of the kind of issues that arise in production contexts.

For each repository, I browsed the Move source files looking for code snippets that contained clear, isolated instances of one of the six defined style categories. The key constraint during collection was that each sample should contain one primary labelled style issue. A snippet labelled with multiple competing issues creates an ambiguous evaluation target: it is unclear whether the model is expected to address all of them, just one, or which one takes priority. Anchoring each sample to one primary labelled issue eliminates this ambiguity and makes the evaluation cleaner. To support reproducibility, the repository cloning and the initial pattern-based scanning of .move files for candidate snippets were carried out using a set of Python scripts, which are published separately together with the category definitions and annotation rules applied during collection (Zerkidis, 2026b).

Each sample was stored in a three-column spreadsheet. The first column contains the code snippet itself, extracted directly from the source repository. The second column contains the label in the format `StyleError/[Category]`, using the six category names defined in the previous section. The third column contains a one to two sentence reference feedback comment explaining the specific issue and the correct form. This Output/Fix column represents the ground truth against which Code Llama's output is evaluated. It is not the corrected code and it is not generated by the model; it is the expected natural-language response to the sample, written to the same standard one would expect from a code reviewer familiar with Move conventions.

3.3.2 Annotation and validation

Several quality checks were applied during collection. Snippets were required to contain syntactically valid Move code, or at minimum to represent a plausible real-world fragment. Samples that duplicated one another on both code and label were excluded. In a small

number of cases, nine snippets affecting eighteen of the 368 samples, a single piece of code exhibited two distinct style issues, typically a missing documentation comment combined with an indentation violation. These snippets were included once per issue with separate labels, to allow each violation to be evaluated independently against the reference feedback for its category. The one-issue-per-sample constraint applies to the labelled issue and the feedback, and does not prohibit the underlying code from containing a second independent issue under a different label. Each sample was reviewed against its label to confirm that the annotated issue was present, that it was the most prominent style concern visible in the snippet, and that the feedback comment described the issue using the correct Move convention. Samples where it was unclear whether a convention violation was intentional were excluded.

Once annotation was complete, the full dataset of labels and feedback comments was submitted to a representative from the Sui Foundation for review, as part of the ongoing collaboration between the academic programme and Sui. The reviewer verified that the style conventions referenced in the ground truth matched accepted Move and Sui ecosystem practices. All 368 samples were approved without modification, which provides external validation that the annotation reflects genuine community standards rather than idiosyncratic interpretation.

The complete annotated dataset, including all 368 code samples with their category labels and reference feedback comments, is archived on Zenodo with DOI 10.5281/zenodo.19682589 (Zerkidis, 2026a). Access is currently under embargo as the dataset forms part of an ongoing Sui Academic Research collaboration; full public release will follow at the end of the embargo period.

The final dataset contains 368 samples distributed across the six categories. `StyleError/Spacing` accounts for 77 samples; `StyleError/MissingDocumentation` for 76; `StyleError/IncorrectIndentation` for 70; `StyleError/InconsistentNaming` for 66; `StyleError/InconsistentFunctionSignature` for 46; and `StyleError/InconsistentImportOrdering` for 33. The distribution reflects the natural occurrence of these issues in real repositories rather than an enforced balance.

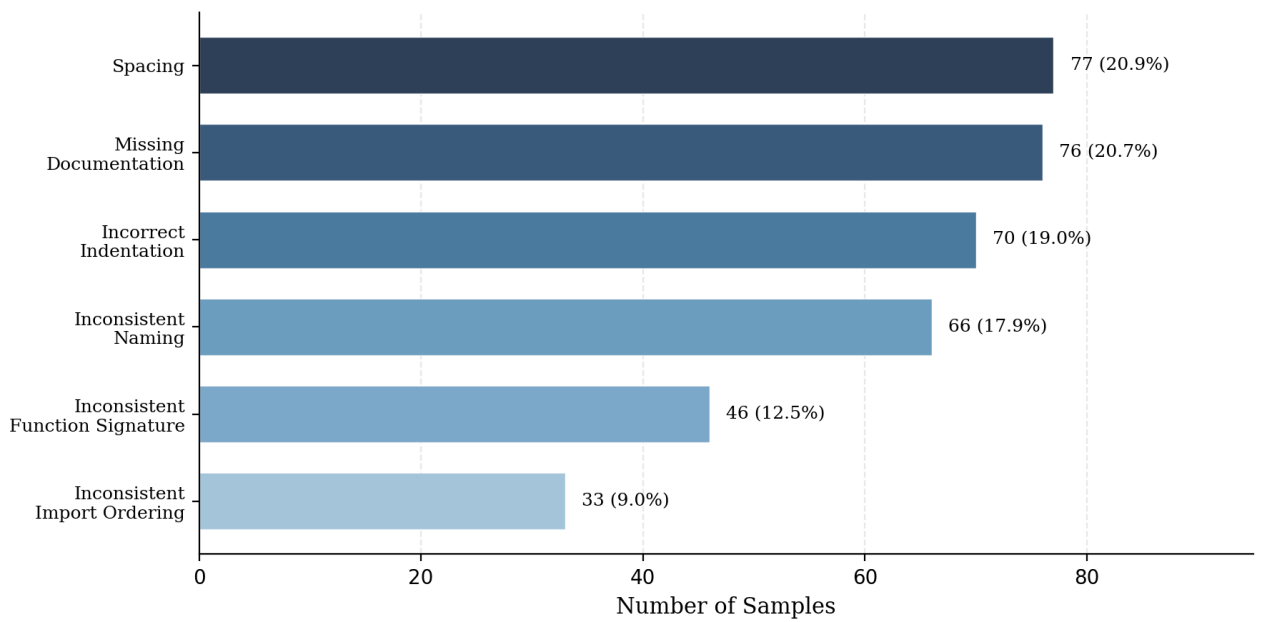


Figure 3.1: Dataset distribution by style error category.

Two samples illustrate the annotation format concretely. The first, labelled `StyleError/InconsistentNaming`, contains three consecutive constant declarations. The Sui-reviewed feedback states: "Naming convention issue: rename `ENotOneTimeWitness` to `E_NOT_ONE_TIME_WITNESS` to follow `E_ + UPPER_SNAKE_CASE` for error constants." The second sample, labelled `StyleError/InconsistentImportOrdering`, shows a block of use statements out of order. The ground truth states: "Import organization issue: group `std::type_name` imports before `sui::bag` imports, separated by a blank line." Both examples illustrate what the annotation format is designed to achieve: a specific, actionable statement about the exact issue present, against which model output can be directly compared.

Table 3.1: Dataset distribution by style error category.

Category	Sample Count	Percentage
StyleError/Spacing	77	20.9%
StyleError/MissingDocumentation	76	20.7%
StyleError/IncorrectIndentation	70	19.0%
StyleError/InconsistentNaming	66	17.9%
StyleError/InconsistentFunctionSignature	46	12.5%
StyleError/InconsistentImportOrdering	33	9.0%
Total	368	100%

3.4 Code Llama Setup and Prompt Design

Code Llama was run locally using Ollama (Ollama, 2024), an open-source tool that allows large language models to be downloaded and executed on local hardware without a commercial API. This approach has two practical advantages. First, it removes any dependency on external API availability, rate limits or cost which matters when processing several hundred samples. Second, it ensures reproducibility: the same model weights and configuration are used for every sample without any variability introduced by a cloud provider's infrastructure or load balancing.

I used the 7B instruction-following variant of Code Llama, specifically the `codellama:instruct` model as made available through Ollama. Instruction-following models are fine-tuned to respond to direct natural-language instructions rather than simply completing code sequences, which makes them more suitable for a feedback generation task. The base model, by contrast, is optimised for code completion and would not reliably produce the kind of natural-language feedback this study requires. No quantisation or configuration changes were applied beyond the defaults provided by the Ollama runtime.

No few-shot examples were included in the prompt. Including examples would have shifted the evaluation from testing the model's baseline capability to testing its ability to imitate provided demonstrations. Since the research question is what Code Llama can do without task-specific guidance, the prompt was kept minimal. It identifies the task, names the six style error categories, presents the code snippet, and asks for specific feedback on any style issues present.

The prompt used for every sample followed this structure:

```
You are a code reviewer specialising in Move smart contracts.
The following Move code may contain one or more of these style
issues: InconsistentNaming, Spacing, IncorrectIndentation,
MissingDocumentation, InconsistentImportOrdering,
InconsistentFunctionSignature. Review the code and provide
specific feedback on any style issues you find. CODE: [code
snippet].
```

This template was held constant for all 368 samples, with only the code snippet varying.

Naming all six categories in the prompt was a deliberate design decision. The alternative would have been to ask the model to identify style issues without specifying the categories, producing more open-ended responses. This was rejected for two reasons. First, unguided

responses are harder to evaluate against category-specific ground truth. Second, telling the model which categories exist frames the task in the same terms used for annotation, creating a fairer comparison.

The entire pipeline from dataset to output was automated using Claude Code (Anthropic, 2025). For each of the 368 samples, Claude Code passed the code snippet to the Ollama API running locally, captured the model's complete text response, and appended it to a results file alongside the original code and label. This automation ensured that every sample was processed under identical conditions, with the same prompt template, the same model configuration, and no human intervention between runs.

3.5 Evaluation Approach

The evaluation asks three questions about each model output. First, does the output correctly identify the category of the style error present in the sample? Second, is the feedback the model provides accurate and specific enough to be useful to a developer? Third, taken together, does the output represent a correct and helpful response to the sample? These three criteria map onto what an effective style-checking tool would need to do: identify the right problem, explain it accurately, and do so in a way that is actionable.

Category identification is the most objective of the three criteria. Each sample has a single label, and the model's output either addresses that category or it does not. An output that discusses `InconsistentNaming` when the label is `InconsistentNaming` counts as a correct identification, regardless of whether the model also mentions other issues. This binary framing is intentional: the evaluation tests whether the model can recognise the relevant issue, not whether it produces a clean single-point response.

Feedback quality is more nuanced. Even when a model correctly names the category, it may give incorrect or irrelevant advice about how to fix the issue. A response that advises renaming `Move` functions to `camelCase`, for instance, gives wrong guidance even if it correctly identified naming as the issue, since `Move` convention requires `snake_case`. Feedback was evaluated as accurate when the specific advice given was correct for `Move`, meaning it aligned with the conventions defined in the style rules section.

Overall correctness combines category identification and feedback quality into a single judgment. A response was considered fully correct when it identified the right category and provided accurate, specific guidance. A response was considered partially correct when it

identified the right category but offered generic, imprecise, or partially inaccurate advice. A response was considered incorrect when it failed to identify the labelled category or when the core advice it gave was wrong for Move.

The evaluation combined automated and manual scoring. Automated scoring was applied to all 368 model outputs, using keyword and pattern matching to compare each response against the reference feedback. A response was scored as correct if it matched the category-specific conventions of the reference, for example the triple-slash doc comment marker for `MissingDocumentation`, `UPPER_SNAKE_CASE` with the required `E_` prefix for error constants under `InconsistentNaming`, four-space indentation for `IncorrectIndentation`, and the standard-before-framework grouping rule for `InconsistentImportOrdering`. A response was scored as partial if it identified the general area without matching the specific convention. A subset of outputs, selected to be representative across categories, was additionally reviewed manually to inform the qualitative analysis presented in section 4.3 and the case studies in section 4.4. This hybrid approach follows the methodology used by Song et al. (2024) in their evaluation of MoveScan, which compared tool output to expert-written expected results, and by Karanjai et al. (2025), who manually assessed the quality of LLM-generated vulnerability repair suggestions for Move code. A fully automated metric such as BLEU score or exact-match comparison was considered and rejected: style feedback can be expressed correctly in many different ways, and surface-level string similarity is a poor proxy for semantic accuracy.

Table 3.2: Scoring rules applied per category during automated evaluation.

Category	Strict (correct)	Partial	Wrong
MissingDocumentation	Response includes the /// triple-slash doc comment syntax	Identifies missing docs but does not specify ///	No mention of documentation
InconsistentNaming	Response gives the correct case form (<code>UPPER_SNAKE_CASE</code> with <code>E_</code> prefix for error constants, <code>snake_case</code> for functions)	Identifies a naming issue but suggests the wrong case form	No mention of naming
IncorrectIndentation	Response recommends four-space indentation	Mentions indentation but recommends a different size	No mention of indentation

Spacing	Response identifies the specific operator or location flagged in the reference	Mentions spacing in general terms but not the specific location	No mention of spacing
InconsistentImportOrdering	Response describes grouping standard library imports before framework imports with a blank line separator	Mentions import order but not the specific grouping rule	No mention of imports
InconsistentFunctionSignature	Response specifically recommends breaking a long signature across multiple lines with parameter alignment	Mentions formatting or readability without describing the line-break fix	No mention of signature formatting

Evaluation was conducted on all 368 samples. Results were recorded per sample and then aggregated by category to identify where the model performed consistently and where it struggled. The per-category analysis is the primary unit of comparison because the research questions ask specifically which categories Code Llama handles well and which it does not.

4. Results and Evaluation

4.1 Dataset Overview

An evaluation of the 368 code samples reveals that not all six style categories are represented evenly. The two largest categories, Spacing and MissingDocumentation, account for over 40 per cent of all code samples evaluated, at 77 and 76 samples respectively, while InconsistentImportOrdering accounts for the lowest number of samples at 33.

Import ordering violations are much lower in volume than spacing errors, because a large percentage of projects in the Sui ecosystem have small import sections containing only a few use statements, where the chance of an import ordering violation is minimal and likely to be missed during review. Spacing errors, by contrast, accumulate with every operator and every function signature written in the same module. A module with 50 function definitions has 50 places where a spacing error could occur, whereas the same module would have only one import block. Similarly, every public function without documentation counts as a sample for MissingDocumentation, so the volume of Spacing and MissingDocumentation samples is comparable for related reasons.

The number of samples for InconsistentFunctionSignature is smaller, at 46, because these errors only become apparent in functions that have multiple parameters or type constraints. Application-level Move code typically has shorter function signatures than infrastructure-level code, and because there are far more application repositories than infrastructure repositories in the Sui ecosystem, InconsistentFunctionSignature errors are at the lower end of the totals. All of the code samples were processed through Code Llama according to the prompt and pipeline described in the Methodology chapter, and the output from each code sample was scored against the Sui-reviewed feedback.

4.2 Quantitative Results

Code Llama produced a response for all 368 samples, and each response was scored against the same criteria used for the Sui-reviewed feedback. A Code Llama output that correctly identified a style error and provided a change consistent with Move convention was considered correct. If an output identified the correct general area but provided vague or incomplete advice, or applied advice from a different programming language, it was considered a partial response. If an output failed to identify a specific issue, or provided

advice that contradicted Move conventions, it was scored as incorrect. The results of this scoring are presented in Table 4.1.

Table 4.1: Code Llama accuracy across all 368 samples, scored against Sui-reviewed feedback.

Category	Samples	Correct	Partial	Wrong	Strict Acc.	Lenient Acc.
IncorrectIndentation	70	70	0	0	100.0%	100.0%
InconsistentImportOrdering	33	14	9	10	42.4%	69.7%
InconsistentNaming	66	19	12	35	28.8%	47.0%
Spacing	77	6	69	2	7.8%	97.4%
MissingDocumentation	76	1	72	3	1.3%	96.1%
InconsistentFunctionSignature	46	0	45	1	0.0%	97.8%
Total	368	110	207	51	29.9%	86.1%

Strict accuracy requires advice matching the approved Move convention. Lenient accuracy includes partial matches where the correct category was identified but the advice was generic or language-inappropriate.

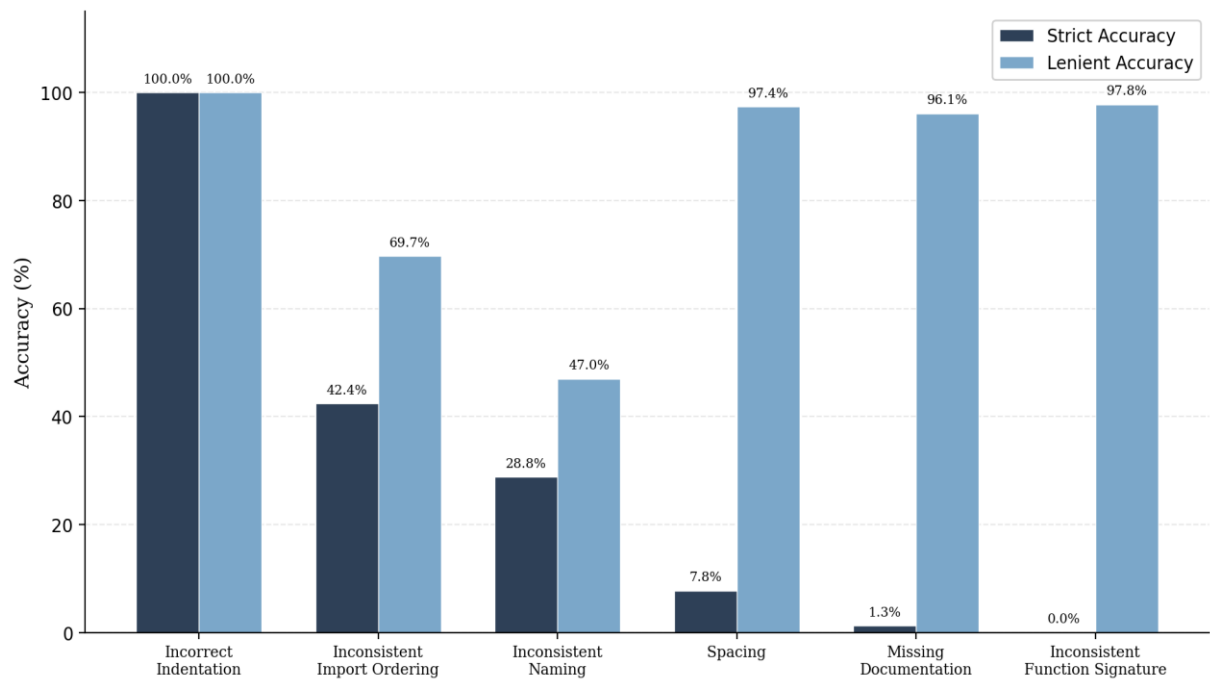


Figure 4.1: Strict versus lenient accuracy by style error category.

The overall strict accuracy of 29.9 per cent means that less than one in three responses gave feedback that a Move developer could act on without correction. The lenient accuracy of 86.1 per cent tells a different story: the model detects that something is wrong in the right general area most of the time but lacks the Move-specific knowledge to say what the correct form should be.

The per-category variation is stark. `IncorrectIndentation` achieves 100 per cent strict accuracy, the only category where the model's advice fully matches the Sui-reviewed feedback. This is not because indentation is an easy task in general; it is because four-space indentation is a near-universal convention across programming languages. The model does not need Move-specific knowledge to get this right. An artefact of the dataset storage process should be noted alongside this result: the spreadsheet format used to hold the samples did not preserve leading whitespace, so the model received each indentation sample with a flattened layout rather than the original violation pattern. The 100 per cent accuracy therefore reflects the model's consistent recommendation of four-space indentation matching the reference convention, rather than precise diagnosis of the specific original violation. This does not affect the dataset's utility for downstream training, where the reference feedback carries the supervised signal, but it is the honest reading of what the strict accuracy figure measures for this category.

At the other end, `InconsistentFunctionSignature` scores 0 per cent strict accuracy despite 97.8 per cent lenient accuracy. The model recognises that function signatures have formatting issues in almost every case but does not produce wording that strictly matches the reference feedback. The reference feedback for this category is itself relatively generic, pointing to the need for proper spacing and line breaks for readability rather than articulating a Move-specific convention that distinguishes it from general formatting advice in any language. The 0 per cent strict accuracy should therefore be read partly as a signal that the model's phrasing of readability advice does not match the reference phrasing under keyword matching, rather than as evidence of a Move-specific knowledge gap comparable to the ones visible elsewhere. `MissingDocumentation` sits in a different place: 1.3 per cent strict accuracy against 96.1 per cent lenient. The model consistently identifies that documentation is missing but almost never specifies that Move uses the `///` doc comment format.

4.3 Qualitative Findings

The most consistent pattern across all 368 outputs is what might be called scattershot feedback: regardless of which single style category a sample represents, Code Llama produces a numbered list touching on all six categories defined in the prompt. A sample labelled `InconsistentFunctionSignature` receives feedback on naming, spacing, indentation, documentation, imports, and function signatures. A sample labelled `InconsistentImportOrdering` receives the same six-point structure. This template pattern appeared in 351 of the 368 responses (95.4 per cent). The model has learned that a code review response should cover multiple dimensions. And it applies that template uniformly without regard for what is actually present in the code.

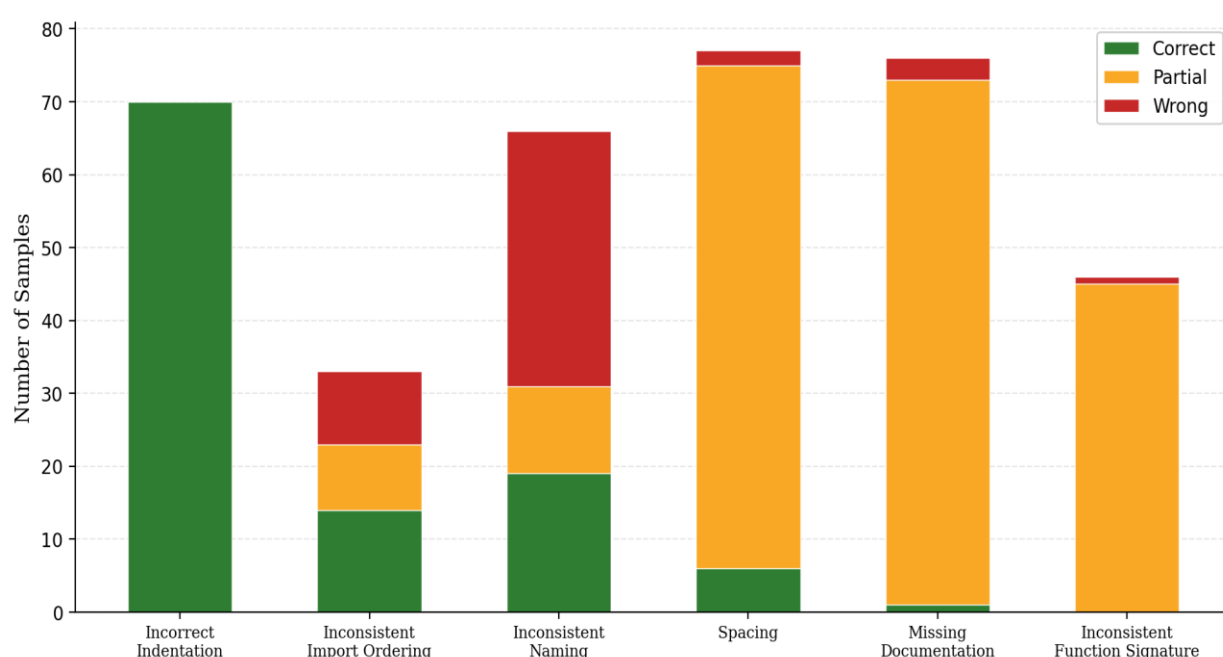


Figure 4.2: Response quality distribution by category (correct, partial, and wrong).

Beyond the structural pattern, the outputs reveal specific failure modes tied to Move's conventions. Across the 66 naming samples, the model repeatedly advises that function names should use `camelCase`. This is the convention in many languages, but Move requires `snake_case` for function names and `E_UPPER_SNAKE_CASE` for error constants. The advice is not a minor imprecision; it points developers in the wrong direction. In 25 of the 66 naming samples, the model explicitly recommended `PascalCase` or `camelCase` for identifiers that should use `UPPER_SNAKE_CASE` according to the Sui-reviewed feedback. In 33 of the 368 total responses (9 per cent), the model explicitly referenced Solidity conventions when analysing Move code.

For the indentation samples, the model correctly identifies that indentation is inconsistent in all 70 cases, achieving 100 per cent strict accuracy. This is the standout category. The reason is straightforward: four-space indentation is not a Move-specific convention. It is standard across Python, Java, Rust, and most modern languages.

The spacing category tells the opposite story. In 69 of 77 samples, the model mentions spacing as an issue but gives generic advice rather than identifying the specific operator or location flagged in the ground truth. Only 6 responses matched the specific fix. The model knows that spacing matters but cannot pinpoint the exact violation in context.

The import ordering responses show moderate performance. The model correctly states in many cases that imports should be grouped and ordered, achieving 42.4 per cent strict accuracy. The problem is that Move's specific grouping rule, which places `0x1 : :` standard library imports before `sui::` framework imports with a blank line between them, is articulated in only a subset of responses.

MissingDocumentation achieves 96.1 per cent lenient accuracy but only 1.3 per cent strict. The model consistently and correctly identifies that public functions lack documentation and recommends adding it. The advice is generic rather than Move-specific: almost no responses mention the `///` triple-slash convention that Move uses. A developer who followed the advice would be doing the right thing, but they would not know the correct format without looking it up.

4.4 Case Studies

Case Study 1: Correct Detection with Generic Advice

One MissingDocumentation sample shows a public test utility function, `new_rewarder_for_test`, which constructs a `Rewarder` struct from two `u128` emission parameters and a generic coin type, with no preceding documentation. The Sui-reviewed feedback states: “Documentation issue: add `///` doc comment above public function `new_rewarder_for_test()` describing its purpose and parameters.” The code appears in the source as follows:

```
#[test_only]
public fun new_rewarder_for_test<CoinType>(
    emissions_per_second: u128,
    growth_global: u128,
): Rewarder {
```

```
// function body omitted  
}
```

Listing 4.1: Excerpt from a MissingDocumentation sample. Source: Zerkidis (2026a), doi: 10.5281/zenodo.19682589.

Code Llama's response includes, among its standard multi-point list, the following under point four: "There is no documentation for the function, which can make it difficult for other developers to understand what the function does and how to use it. It's recommended to add documentation for all public functions in Move." This response correctly identifies the issue and gives advice that, while generic, is accurate.

The reason this case succeeds is the language-agnostic nature of the task. An absent documentation comment is visually obvious from the code structure, and the correct remedy does not require knowing anything specific about Move. The model does not need to know that Move uses `///` rather than Javadoc syntax to give a response that points in the right direction.

Case Study 2: A Complete Failure

One Spacing sample contains a native function declaration in which the type constraint expression uses `copy+ drop+ store` instead of `copy + drop + store`, with the plus signs missing surrounding spaces. The Sui-reviewed feedback is specific: "Spacing issue in function add: add spaces around + in `copy+ drop` to read `copy + drop`." The code appears in the source as follows:

```
native public fun add<T0: drop, T1: copy+ drop+ store, T2:  
store>(   
    a0: T0,   
    a1: &mut WitTable<T0, T1, T2>,   
    a2: T1,   
    a3: T2,   
);
```

Listing 4.2: Excerpt from a Spacing sample. Source: Zerkidis (2026a), doi: 10.5281/zenodo.19682589.

Code Llama's response makes no mention of spacing. Instead, it focuses entirely on naming: the model suggests renaming the `add` function to `insert` or `put`, the `keys` function to `get_keys` or `list_keys`, and the `borrow` function to `get` or `fetch`. None of this is related to the actual issue in the code, and the renaming advice is itself wrong, since `add`, `keys`, and `borrow` are conventional names in Move collection interfaces.

The spacing issue appears inside angle brackets in type constraint notation, a syntactic pattern specific to Move. A model with minimal exposure to Move's type constraint syntax would not recognise the spacing convention within that notation and, when scanning the code for problems, would latch onto something else.

4.5 Discussion

The answers to the four questions posed in the introduction are discussed in this section, using the entire dataset to illustrate each answer.

4.5.1 Answering the research questions

The first research question asked whether Code Llama provides accurate style error detection for Move smart contracts. The answer is mixed. From a broad perspective, Code Llama indicates that a style error may be present 86.1 per cent of the time. From a stricter viewpoint, however, its recommended solutions are consistent with the Sui-approved Move convention only 29.9 per cent of the time. The model can see potential errors, but it does not consistently provide accurate recommendations that are compatible with the Move programming language.

The second research question examined how the model's recommended solutions compared to those provided by the Sui review. There are significant differences between the two. The Sui reviews provide specific information in a detailed format of one to two sentences, while Code Llama provides generic information in a broad format covering all style error categories regardless of the specific error identified. The recommendations provided by Code Llama were typically neither specific nor accurate. For the style error category of `IncorrectIndentation`, however, the recommendations provided by the model were consistent with the Sui-approved recommendations. As a result, a developer would have to perform their own research before using any recommendations outside `IncorrectIndentation`.

The third research question focused on which style error categories are handled most and least effectively by the model. Code Llama is highly effective at detecting `IncorrectIndentation` errors, at 100 per cent strict accuracy. It is moderately effective at identifying `InconsistentImportOrdering` errors, at 42.4 per cent strict accuracy, and less effective at identifying `InconsistentNaming` errors, at 28.8 per cent strict accuracy. Although Code Llama is generally effective at detecting style errors related to Spacing,

MissingDocumentation, and InconsistentFunctionSignature, the recommendations it provides for these categories are in many cases inaccurate. The model can identify issues in code it has not been trained on without having any information about the Move programming paradigm, but because of the lack of Move-specific training data, it cannot accurately identify Move's conventions for naming functions, writing comments, or ordering imports.

The fourth research question considered whether Code Llama is a good tool for developing Move applications. To be blunt, Code Llama is not yet a good tool for enforcing Move style guidelines. Approximately 29.9 per cent of the recommendations made by Code Llama are accurate, which means developers would have to correct roughly two thirds of the suggestions offered by the model. In particular, the fact that the model consistently recommends camelCase as the naming style for functions in Move code shows how the model's inaccuracies could result in style violations.

There is, however, a caveat that may justify limited use of Code Llama, based on its leniency rate of 86.1 per cent. Although it is an imperfect tool at this time, Code Llama has limited value in that its ability to detect style issues within a code snippet makes it possible to identify whether a style problem exists, even though the model does not provide verifiable remedies. In the absence of a dedicated style linting tool, the ability to receive an imperfect detection signal has value, provided that Move developers are aware of Code Llama's limitations and verify its advice against Move's style conventions independently.

4.5.2 Comparison with prior work

The performance observed in this work is much lower than that of other studies which performed similar tasks with greater resources. For example, Wadhwa et al. (2024) report that their CORE system resolved approximately 76.8 per cent of Java code quality issues, while Xia et al. (2023) report very high resolution rates for pattern-based bugs in both Python and Java. These works cannot be directly compared with the present study, because both tested code correction rather than feedback generation, and both used languages that have far greater representation in LLM training data. The comparison still gives a useful sense of scale.

Karanjai et al. (2025) provide the most relevant comparison, because they assessed Code Llama and other models on the task of repairing Move vulnerability issues. Their results showed that the performance of the models on Move was consistently lower than on

Solidity, which they attributed to Move being underrepresented in the training data. The results of this study confirm that conclusion and add a further level of detail. The performance of the model was not uniformly low across all Move tasks. Instead, it dropped sharply for tasks that required a Move-specific standard.

The pattern-based versus comprehension-based distinction introduced in the literature review does not explain the results in the way that was predicted at the start of this study. The expectation was that pattern-matching tasks such as spacing and naming would perform better than comprehension-based tasks such as documentation generation. The actual results are more complex. `IncorrectIndentation`, a pattern-matching task, achieved 100 per cent accuracy, whereas `Spacing`, also a pattern-matching task, achieved only 7.8 per cent, and `InconsistentNaming`, also a pattern-matching task, achieved only 28.8 per cent. The axis that best predicts performance appears to be programming language specificity rather than the difficulty of the pattern. Tasks where the correct answer is the same regardless of the programming language tend to produce good performance, while tasks that require knowledge of a Move-specific standard tend to produce poor performance. A related observation is that the gap between strict and lenient accuracy for a category reflects not only the model's knowledge of the Move specification but also the specificity of the reference feedback. `MissingDocumentation` and `InconsistentNaming` use a specific Move convention in the reference feedback, the triple-slash marker and the `E_` prefix with `UPPER_SNAKE_CASE`, and the gap there indicates a genuine knowledge deficit. `InconsistentFunctionSignature` uses a more generic encoding of its reference feedback, so the gap there partly reflects a difference in phrasing rather than a knowledge deficit.

The deeper implication is that this gap is not an essential property of the LLM approach but a consequence of the model's training data. A model fine-tuned on Move code with style annotations would be starting from a much stronger position. The present study establishes what the zero-adaptation baseline looks like, and the distance between that baseline and reliable style enforcement is a measure of how much domain-specific training would help.

5. Conclusion

5.1 Summary of Contributions

This study set out to answer a practical question: can Code Llama, applied without modification or fine-tuning, detect style errors in Move smart contracts and generate feedback accurate enough to be useful? The answer is qualified. The model detects issues reliably (86.1 per cent lenient accuracy) but gives Move-correct advice far less often (29.9 per cent strict accuracy), with performance splitting cleanly along the language-specificity axis described in the discussion.

The contribution of this work is twofold. The first is the dataset: 368 annotated Move code samples drawn from real Sui ecosystem repositories, covering six style error categories, each labelled with a category identifier and a reference feedback comment reviewed by the Sui Foundation. No equivalent resource existed before this study. All prior LLM evaluation on Move code, most notably Karanjai et al. (2025), focused on security vulnerabilities rather than style, and used different annotation formats suited to correction tasks rather than feedback generation. The dataset produced here fills that gap and provides a foundation for future evaluation work on Move code quality. The dataset is archived on Zenodo with DOI 10.5281/zenodo.19682589 (Zerkidis, 2026a), under embargo for the duration of the ongoing Sui Academic Research collaboration.

The second contribution is the evaluation itself. The finding that a general-purpose coding model without fine-tuning performs reliably only on language-agnostic tasks is not, on reflection, surprising, but it needed to be established empirically for Move specifically. The key insight is that the predictive axis for LLM performance on style tasks is not pattern complexity but language-specificity: the model performs well when the correct answer is the same regardless of programming language and poorly when the correct answer requires knowing a Move-specific convention. The results confirm and extend the pattern identified by Karanjai et al. (2025) for vulnerability repair: Move's underrepresentation in LLM training corpora creates a consistent performance ceiling that applies across task types, not just security ones.

5.2 Limitations

Several limitations affect what can be concluded from these results.

The automated scoring component used to evaluate the model's performance relies on keyword and pattern matching against the reference feedback provided by the Sui Foundation reviewer. This automated method can be less reliable in edge cases where the model produces a correct answer using unexpected phrasing or structure. Although the reference feedback itself was validated by a Sui Foundation reviewer without modification, the programmatic matching between the model's outputs and the reference did not involve a manual comparison of each individual output. A full manual review of all 368 model outputs would yield a more accurate picture of their accuracy. Another limitation involves the use of a spreadsheet format to store the dataset for the IncorrectIndentation category. The spreadsheet format did not preserve leading whitespace, so the input samples for indentation were in a flattened form rather than the original violation format. The loss of leading whitespace does not invalidate the reference feedback or reduce the usability of the dataset for future training, but it does mean that the 100 per cent accuracy in this category reflects a consistent default response that matches the reference feedback rather than a precise diagnosis of the incorrect indentation.

The evaluation focuses on a single model, Code Llama. Code Llama has the advantages of being open-source, of being able to operate without an expensive commercial API, and of being a credible option for a developer who wants to implement an LLM-based style tool without institutional resources. The rationale for selecting Code Llama is sound, but the results give no indication of how the same task would be performed by competing models such as GPT-4, DeepSeek, or Gemini.

The annotations in this study were performed by a single researcher, and the complete set of labels and reference corrections was then sent to the Sui Foundation for confirmation, with no changes made. In this case, review by an expert in the area should be considered more valuable than measuring inter-rater reliability between two student annotators, because the expert review was made against working knowledge of best practice in the ecosystem rather than against a second interpretation of the same material. This limitation could be further reduced by adding a second-round multi-annotator selection process before review.

Finally, this study examines only the baseline ability of Code Llama. The results show what Code Llama is capable of without task-specific adaptation, which is of real importance for companies that want to adopt it easily into their workflows. However, it is not known what

improvements could be made if Code Llama were trained specifically for Move. Move's limited presence in Code Llama's training data is a language-level constraint rather than an upper limit on the ability of LLMs to enforce Move style.

5.3 Future Work

The most immediate continuation of this project is the fine-tuning stage, which was the original purpose for collecting the dataset. This work establishes a foundation for future developer activity within the Sui community as part of an academic collaboration. In addition to supporting other academics who are creating tools for evaluating Move smart contracts, one intended outcome of this project is an improved version of the current Code Llama tool, trained using only Move-specific samples. A separate research effort would involve training a model using only the samples annotated in this study, which would allow the development of a Move-specific model containing Move-specific coding conventions, so that users could analyse Move code more accurately than they can with the unmodified Code Llama. Models trained without access to Move-specific knowledge are likely to yield low strict accuracy, below 50 per cent, in categories that require specialised knowledge of Move coding conventions.

A useful benchmark beyond Code Llama would be to conduct multi-model evaluations on the same 368 samples. Performing multi-model evaluations would determine whether the performance patterns observed with Code Llama correlate with larger trends in LLM performance on underrepresented languages. The work of Karanjai et al. (2025) offers a framework for this type of multi-model comparison in the domain of vulnerability repair, and performing a similar multi-model evaluation for code style enforcement would provide a more complete view of the relative performance of different LLMs on Move code.

A longer-term direction is building a dedicated Move linter informed by the style rules defined in this work. The six categories examined here include the most common and consequential style issues found in Sui ecosystem code. The reference feedback comments in the dataset are specific and precise enough to serve as a specification for an automated implementation of these rules. There is currently no tool in the ecosystem that checks for these conventions, and one based on explicit rules would be more reliable, and less prone to model variance, than an LLM applied as-is, and could serve as a basis against which future LLM-based tools are compared.

As the developer community for the Move programming language continues to grow, the quality and robustness of the tooling needed by developers will become increasingly important. This study represents a first step towards establishing a framework for those tools and developing a consistent coding experience for developers building in the Sui ecosystem. Building those tools is the path ahead.

Self-Reflection

I started this project with little experience in blockchain development besides some theoretical knowledge. Reading about Move and reading production-ready Move code are two different activities, as well. The difference in programming models for resource ownership and resource type has a direct impact on how developers will structure their programs. In Move, functions return a resource instead of just a value, and it is up to the caller to consume, transfer or store that resource. I was surprised at how long it took me to become comfortable enough with the ownership and resource type system in Move to be able to tell when a snippet was violating a convention. I found myself viewing many real code examples over and over until I could understand the various patterns.

Collecting a dataset was much more difficult than I anticipated. The repositories I wanted to look at were well-maintained open source ones, as they are representative of actual production practices and therefore have a consistent style. Most style violations come from experimental or unmaintained repositories, which do not give a good indication of what the dataset is interested in studying. To locate 368 actual code snippets with isolated style issues, I had to review many more repositories than are represented in the dataset, and reject many more samples than I kept. The single-issue-per-sample constraint that I used as part of my methodology was the right choice, but it caused me to take longer to gather data.

Automating the evaluation pipeline using Claude Code gave me a different viewpoint about the Code Llama results than I would have had if I had only looked at the results produced by Code Llama. Watching an AI tool build out the testing infrastructure for another AI tool, and then reading the outputs produced by Code Llama, helped me to understand the gap between the two systems, which is something I could not get just from reading about LLM limitations. I found Code Llama to be very consistent in its application of the same six-point template to completely different code snippets. The consistency in this case provided me with insight into the way that instruction-following models respond to prompts that enumerate all possible categories, and it altered the way I viewed the evaluation design in retrospect.

If I could start over again, I would want to ensure that at least two models were evaluated, in order to determine whether the performance patterns obtained in this study were specific to Code Llama or whether something broader applied to LLMs and underrepresented programming languages. The strict and lenient accuracy split emerged as the key finding

during analysis rather than being something I designed the evaluation around from the outset. The variation between lenient accuracy (86.1 per cent) and strict accuracy (29.9 per cent) was the most distinctive finding of the study, and if I had built a rubric specifically around that variation from the start, I would have generated cleaner category-level data.

The project developed additional skills that I would not have been able to gain through coursework alone. The process of creating and applying my own annotation criteria to hundreds of code samples, and then evaluating the output of a model based on my created criteria, presents a set of experiences that can only be gained through hands on experience. Reading about empirical research methodology is very different from actually doing it.

References

- Anthropic (2025) Claude Code. Available at: <https://www.anthropic.com/claude-code> (Accessed: 23 May 2026).
- Bartoletti, M., Benetollo, L., Bugliesi, M., Crafa, S., Dal Sasso, G., Pettinau, R., Pinna, A., Piras, M., Rossi, S., Salis, S., Spanò, A., Tkachenko, V., Tonelli, R. and Zunino, R. (2025) ‘Smart contract languages: A comparative analysis’, *Future Generation Computer Systems*, 164, Article 107563. doi: 10.1016/j.future.2024.107563.
- BeosinBlockchainSecurity (2023) Move Lint. Available at: <https://github.com/BeosinBlockchainSecurity/Move-Lint> (Accessed: 14 March 2026).
- Blackshear, S., Cheng, E., Dill, D.L., Gao, V., Maurer, B., Nowacki, T., Pott, A., Qadeer, S., Rain, D., Russi, S., Sezer, T., Zakian, R. and Zhou, R. (2019) Move: A Language With Programmable Resources. Available at: <https://diem-developers-components.netlify.app/papers/diem-move-a-language-with-programmable-resources/2019-06-18.pdf> (Accessed: 6 January 2026).
- Booger, C. and Moonen, L. (2008) ‘Assessing the value of coding standards: An empirical study’, in *Proceedings of the 2008 IEEE International Conference on Software Maintenance (ICSM)*. Beijing: IEEE, pp. 277–286.
- Brown, T.B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D.M., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I. and Amodei, D. (2020) ‘Language models are few-shot learners’, *Advances in Neural Information Processing Systems*, 33, pp. 1877–1901.
- Buse, R.P.L. and Weimer, W.R. (2010) ‘Learning a metric for code readability’, *IEEE Transactions on Software Engineering*, 36(4), pp. 546–558. doi: 10.1109/TSE.2009.70.
- Buterin, V. (2014) Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform. Available at: <https://ethereum.org/en/whitepaper/> (Accessed: 8 December 2025).
- Chainalysis (2022) 2022 Biggest Year Ever For Crypto Hacking. Available at: <https://blog.chainalysis.com/reports/2022-biggest-year-ever-for-crypto-hacking/> (Accessed: 8 December 2025).
- Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H.P.O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G. et al. (2021) ‘Evaluating large language models trained on code’, *arXiv preprint arXiv:2107.03374*.
- Danezis, G., Kokoris-Kogias, L., Sonnino, A. and Spiegelman, A. (2022) ‘Narwhal and Tusk: A DAG-based mempool and efficient BFT consensus’, in *Proceedings of the 17th European Conference on Computer Systems (EuroSys)*. New York: ACM, pp. 34–50.
- De Baets, C., Suleiman, B. and Nour, B. (2024) ‘Vulnerability detection in smart contracts: A comprehensive survey’, *arXiv preprint arXiv:2407.07922*.
- Electric Capital (2024) Crypto Ecosystems. Available at: <https://github.com/electric-capital/crypto-ecosystems> (Accessed: 22 January 2026).
- Feist, J., Grieco, G. and Groce, A. (2019) ‘Slither: A static analysis framework for smart contracts’, in *Proceedings of the 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB)*. Montreal: IEEE, pp. 8–15.

- Giatzis, A., Papangelou, S. and Georgiadis, C.K. (2025) 'A comparative study of Solidity and Sui Move: Advancing smart contract development', *IEEE Access*, 13, pp. 170504–170522. doi: 10.1109/ACCESS.2025.3615072.
- Giatzis, A. and Georgiadis, C.K. (2025) 'A pattern-oriented ontology and workflow modeling approach for the Sui Move programming language', *Information*, 17(1), Article 4. doi: 10.3390/info17010004.
- Hejazi, N. and Lashkari, A.H. (2025) 'A comprehensive survey of smart contracts vulnerability detection tools: Techniques and methodologies', *Journal of Network and Computer Applications*. doi: 10.1016/j.jnca.2025.104096.
- Iuliano, G. et al. (2024) 'Smart contract vulnerabilities, tools, and benchmarks: An updated systematic literature review', *arXiv preprint arXiv:2412.01719*.
- Karanjai, R., Blackshear, S., Xu, L. and Shi, W. (2025) 'Securing Smart Contract Languages with a Unified Agentic Framework for Vulnerability Repair in Solidity and Move', *arXiv preprint arXiv:2502.18515*. Also in *IEEE International Conference on Blockchain* (2025). doi: 10.1109/Blockchain62396.2025.00058.
- Keilty, E. (2023) *Model Checking and Gas Optimization of Move Smart Contracts, and Transaction Order Dependency Detection and Rectification*. MSc thesis. University of Toronto.
- Kupari, A., Giacaman, N. and Terragni, V. (2025) 'Adoption and evolution of code style and best programming practices in open-source projects', in *Proceedings of the 41st IEEE International Conference on Software Maintenance and Evolution (ICSME)*. Auckland: IEEE, pp. 187–198.
- Lawrie, D., Morrell, C., Feild, H. and Binkley, D. (2006) 'What's in a name? A study of identifiers', in *Proceedings of the 14th IEEE International Conference on Program Comprehension (ICPC)*. Athens: IEEE, pp. 3–12.
- Luo, Y., Li, X. and Liu, Z. (2025) 'MoveScanner: Analysis of Security Risks of Move Smart Contracts', *arXiv preprint arXiv:2508.17964*.
- Martin, R.C. (2008) *Clean Code: A Handbook of Agile Software Craftsmanship*. Upper Saddle River, NJ: Prentice Hall.
- McConnell, S. (2004) *Code Complete: A Practical Handbook of Software Construction*. 2nd edn. Redmond, WA: Microsoft Press.
- Minelli, S., Mocchi, A. and Lanza, M. (2015) 'I know what you did last summer: An investigation of how developers spend their time', in *Proceedings of the 23rd IEEE International Conference on Program Comprehension (ICPC)*. Florence: IEEE, pp. 25–35.
- Mysten Labs (2022) *The Sui Smart Contracts Platform*. Available at: <https://docs.sui.io/paper/sui.pdf> (Accessed: 6 January 2026).
- Nakamoto, S. (2008) *Bitcoin: A Peer-to-Peer Electronic Cash System*. Available at: <https://bitcoin.org/bitcoin.pdf> (Accessed: 6 January 2026).
- Ollama (2024) *Ollama*. Available at: <https://ollama.com> (Accessed: 23 May 2026).
- Parity Technologies (2017) *Security Alert: Parity Wallet library contract vulnerability*. Available at: <https://www.parity.io/blog/security-alert-2/> (Accessed: 9 December 2025).
- Patrignani, M. and Blackshear, S. (2021) 'Robust safety for Move', *arXiv preprint arXiv:2110.05043*.

- Pearce, H., Ahmad, B., Tan, B., Dolan-Gavitt, B. and Karri, R. (2022) ‘Asleep at the keyboard? Assessing the security of GitHub Copilot’s code contributions’, in Proceedings of the 43rd IEEE Symposium on Security and Privacy (S&P). San Francisco: IEEE, pp. 754–768.
- Posnett, D., Hindle, A. and Devanbu, P. (2011) ‘A simpler model of software readability’, in Proceedings of the 8th Working Conference on Mining Software Repositories (MSR). New York: ACM, pp. 73–82.
- Prenner, J.A., Babii, H. and Robbes, R. (2022) ‘Can OpenAI’s Codex fix bugs? An evaluation on QuixBugs’, in Proceedings of the 2022 IEEE/ACM International Workshop on Automated Program Repair (APR). Pittsburgh: IEEE, pp. 69–75.
- Rozière, B., Gehring, J., Gloeckle, F., Sootla, S., Gat, I., Tan, X.E., Adi, Y., Liu, J., Sauvestre, R., Remez, T., Rapin, J., Kozhevnikov, A., Evtimov, I., Bitton, J., Bhatt, M., Ferber, C.C., Grattafiori, A., Xiong, W., Defossez, A., Copet, J., Azhar, F., Touvron, H., Martin, L., Hambro, N., Roziere, F., Joulin, A., Grave, E., Lample, G. and Synnaeve, G. (2023) ‘Code Llama: Open foundation models for code’, arXiv preprint arXiv:2308.12950.
- Sadowski, C., Aftandilian, E., Eagle, A., Miller-Cushon, L. and Jaspán, C. (2018) ‘Lessons from building static analysis tools at Google’, Communications of the ACM, 61(4), pp. 58–66. doi: 10.1145/3188720.
- Sobania, D., Briesch, M., Hanna, C. and Rothlauf, F. (2023) ‘An analysis of the automatic bug fixing performance of ChatGPT’, in Proceedings of the 2023 IEEE/ACM International Workshop on Automated Program Repair (APR). Melbourne: IEEE, pp. 23–30.
- Song, S., Chen, J., Chen, T., Luo, X., Li, T., Yang, W., Wang, L., Zhang, W., Luo, F., He, Z., Lu, Y. and Li, P. (2024) ‘Empirical Study of Move Smart Contract Security: Introducing MoveScan for Enhanced Analysis’, in Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA 2024). New York: ACM, pp. 1682–1694. doi: 10.1145/3650212.3680391.
- Sun, J., Hui, Z. and Wang, X. (2023) ‘Move smart contract vulnerability detection based on resource-flow analysis’, in Proceedings of the 10th International Conference on Dependable Systems and Their Applications (DSA). IEEE, pp. 389–398.
- Szabo, N. (1997) ‘Formalizing and securing relationships on public networks’, First Monday, 2(9). doi: 10.5210/fm.v2i9.548.
- Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S. et al. (2023) ‘Llama 2: Open foundation and fine-tuned chat models’, arXiv preprint arXiv:2307.09288.
- Vacca, A., Di Sorbo, A., Visaggio, C.A. and Canfora, G. (2021) ‘A systematic literature review of blockchain and smart contract development: techniques, tools, and open challenges’, Journal of Systems and Software, 174, Article 110891. doi: 10.1016/j.jss.2020.110891.
- van Rossum, G., Warsaw, B. and Coghlan, A. (2001) PEP 8 – Style Guide for Python Code. Available at: <https://peps.python.org/pep-0008/> (Accessed: 15 February 2026).
- Wadhwa, N., Pradhan, J., Sonwane, A., Sahu, S.P., Natarajan, N., Kanade, A., Parthasarathy, S. and Rajamani, S. (2024) ‘CORE: Resolving Code Quality Issues Using LLMs’, Proceedings of the ACM on Software Engineering, 1(FSE), Article 36. doi: 10.1145/3643762.
- Xia, C.S., Wei, Y. and Zhang, L. (2023) ‘Automated program repair in the era of large pre-trained language models’, in Proceedings of the 45th IEEE/ACM International Conference on Software Engineering (ICSE). Melbourne: IEEE, pp. 1482–1494.

- Zhang, Q., Fang, C., Xie, Y., Ma, Y., Sun, W., Yang, Y. and Chen, Z. (2024) ‘A systematic literature review on large language models for automated program repair’, arXiv preprint arXiv:2405.01466.
- Zhang, W., Chen, T., Li, T., Wang, L., Zhu, Z. and Shen, H. (2025) ‘Analyzing and detecting four types of critical security vulnerabilities in move smart contracts’, *Cluster Computing*, 28, Article 633. doi: 10.1007/s10586-025-05439-1.
- Zhong, J.E., Cheang, K., Qadeer, S., Grieskamp, W., Blackshear, S., Park, J., Zohar, Y., Barrett, C. and Dill, D.L. (2020) ‘The Move Prover’, in Lahiri, S.K. and Wang, C. (eds.) *Computer Aided Verification*. Cham: Springer, pp. 137–150.
- Zerkidis, D. (2026a) *Move Style Error Dataset: 368 Annotated Move Code Samples for Style Enforcement Evaluation* [Data set]. Zenodo. doi: 10.5281/zenodo.19682589.
- Zerkidis, D. (2026b) *Move Style Error Data Collection*. [Online] Available from: <https://github.com/deltadevZ/move-style-data-collection>. [Accessed: 21 May 2026].

Appendix A : Ethical Form



Request for ethical approval for students on taught programmes

Please complete this form and return it to your supervisor as advised in your module handbook. Feedback on your application will be via your supervisor or co-ordinator.

Your Name:	Dimitrios Zerkidis		
Student ID:	100778080		
Unimail address:	100778080@unimail.derby.ac.uk		
Other contact information	DZ1206@mc-class.gr		
Programme name and code	BSC (Hons) Computer Science		
Module name and code	Individual Project 6CM995		
Name of supervisor	Ioannis Tsioulis		
Name of co-ordinator	Alexandros Rompas		
Title of proposed research study			
AI-Based Code Style Enforcement with CodeLlama			
Supervisor Comments			
Are the ethical implications of the proposed research adequately described in this application?	Yes ☒	No ☐	
Does the overall study have low, moderate or high risk in terms of ethical implications?	Low ☒	Moderate ☐	High ☐
Does the study method describe a process of research that is ethically sound?	Yes ☒	No ☐	
Signatures			
The information supplied is, to the best of my knowledge and belief, accurate. I clearly understand my obligations and the rights of the participants. I agree to act at all times in accordance with University of Derby Policy and Code of Practice on Research Ethics: http://www.derby.ac.uk/research/uod/ethics/			
Signature of applicant			
Date of submission by applicant	12/11/2025		
Signature of supervisor			
Date of signature by supervisor	21/11/2025		
<i>For Committee Use</i> <i>Reference Number (Subject area initials/year/ID number).....</i>			
Date received..... Date approved Signed..... Comments			

Appendix B : Project Plan Gantt Diagram

